Reviewer-Revised Research Report: Transformation Vectors and Composition Diagnostics

Date

2026-06-12

Purpose

Fix the reviewer-revised research state for external verification. This version incorporates AC-level review concerns: endpoint leakage, syntax-holdout overclaiming, McNemar tests, semantic-control statistics, and the terminology change to a third-order signed permutation coherence test.

Track 1 fixed result

Delta vectors add reproducible information beyond target-only endpoints in the multiseed full-semantic setting. The syntax=1.0 result has now been directly ablated: y_only also reaches 1.0, so the syntax split is interpreted as endpoint/surface leakage rather than as geometry.

Track 2 fixed result

Composition order matters. Semantic equivalence shifts noncommutativity distributions with strong statistical tests. The third-order result is now framed as signed permutation coherence, not a formal Jacobi identity. Decoder replication now supports QMT below-null cancellation for GPT-2 and DistilGPT-2.

Main conclusion

The evidence supports a local QMT signed-permutation cancellation effect across tested encoder models, while negation-heavy triples expose the boundary of the phenomenon. The current work is promising but not submission-ready without the listed blocker experiments.

Delta vs endpoint ablation with McNemar evidence

model	delta_mean	delta_std	y_only_mean	y_only_std	concat_mean	concat_std	x_only_mean	diff_delta_y	diff_delta_concat	max_mcnemar_p	all_mcnemar_sig_0_001
bert-base-uncased	0.9075	0.0117	0.8632	0.0167	0.8969	0.0185	0.1667	0.0443	0.0106	0.0000	1.0000
distilgpt2	0.8572	0.0125	0.7265	0.0094	0.7604	0.0107	0.1667	0.1307	0.0968	0.0000	1.0000
distilroberta-base	0.8810	0.0053	0.8306	0.0056	0.8417	0.0060	0.1667	0.0504	0.0394	0.0000	1.0000
gpt2	0.8332	0.0076	0.7496	0.0152	0.7765	0.0093	0.1667	0.0836	0.0567	0.0000	1.0000
roberta-base	0.8817	0.0090	0.8391	0.0072	0.8666	0.0089	0.1667	0.0426	0.0151	0.0000	1.0000

Syntax representation ablation: y_only solves the split

model	classifier	concat	delta	x_only	y_only
bert-base-uncased	linear_svc	1.0000	1.0000	0.1667	1.0000
bert-base-uncased	logreg	0.9904	0.9910	0.1667	0.9861
distilgpt2	linear_svc	1.0000	1.0000	0.1667	1.0000
distilgpt2	logreg	0.9976	1.0000	0.1667	0.9957
distilroberta-base	linear_svc	1.0000	1.0000	0.1667	1.0000
distilroberta-base	logreg	1.0000	1.0000	0.1667	1.0000
gpt2	linear_svc	1.0000	1.0000	0.1667	1.0000
gpt2	logreg	0.9979	0.9924	0.1667	0.9876
roberta-base	linear_svc	1.0000	1.0000	0.1667	1.0000
roberta-base	logreg	1.0000	1.0000	0.1667	1.0000

Layerwise/pooling syntax sanity check: top rows

classifier	accuracy	model	layer	pooling	representation	n_base	n_train	n_test
linear_svc	1.0000	bert-base-uncased	0.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	0.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	1.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	1.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	1.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	4.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	4.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	3.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	3.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	3.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	cls	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	2.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	12.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	7.0000	mean	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	6.0000	last_token	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	6.0000	cls	delta	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	11.0000	mean	y_only	200.0000	2400.0000	4800.0000
linear_svc	1.0000	bert-base-uncased	11.0000	mean	delta	200.0000	2400.0000	4800.0000

DeBERTa-v3-small modern spot-check

model	classifier	concat	delta	x_only	y_only
microsoft/deberta-v3-small	linear_svc	0.8225	0.8708	0.1667	0.8042
microsoft/deberta-v3-small	logreg	0.8283	0.7958	0.1667	0.7700

Third-order signed permutation summary with bootstrap CI

model	triple	mean_relative_jacobi_norm	relative_jacobi_norm_ci95_	relative_jacobi_norm_ci95_	mean_jacobi_to_null_mean_ratio	jacobi_to_null_mean_ratio_ci9	jacobi_to_null_mean_ratio_ci9	mean_null_percentile_smaller_is	n
bert-base-uncased	NMT	0.3048	0.3005	0.3090	0.7750	0.7669	0.7829	0.3522	100.0000
bert-base-uncased	NQM	0.2695	0.2660	0.2727	0.8628	0.8533	0.8720	0.3850	100.0000
bert-base-uncased	NQT	0.3719	0.3674	0.3763	1.1170	1.1082	1.1259	0.7005	100.0000
bert-base-uncased	QMT	0.2760	0.2712	0.2807	0.6833	0.6773	0.6892	0.1018	100.0000
distilroberta-base	NMT	0.3522	0.3485	0.3558	1.1338	1.1276	1.1404	0.7066	100.0000
distilroberta-base	NQM	0.2847	0.2809	0.2882	1.0821	1.0735	1.0914	0.7777	100.0000
distilroberta-base	NQT	0.2777	0.2752	0.2802	1.0803	1.0765	1.0841	0.6274	100.0000
distilroberta-base	QMT	0.1849	0.1832	0.1866	0.6311	0.6267	0.6356	0.0997	100.0000
roberta-base	NMT	0.4315	0.4255	0.4371	1.2532	1.2423	1.2637	0.8057	100.0000
roberta-base	NQM	0.3122	0.3086	0.3158	0.9799	0.9743	0.9853	0.6475	100.0000
roberta-base	NQT	0.3452	0.3413	0.3490	1.0805	1.0739	1.0871	0.7059	100.0000
roberta-base	QMT	0.2190	0.2160	0.2222	0.6229	0.6168	0.6287	0.1013	100.0000

Decoder signed permutation replication

model	triple	mean_jacobi_norm	mean_relative_jacobi_n	median_null_relative_jacobi	jacobi_jacobi_to_null_mean	null_mean_null_percentile_smaller	n	relative_jacobi_norm_ci95	relative_jacobi_norm_ci95	jacobi_to_null_mean_ratio_ci95	jacobi_to_null_mean_ratio_ci95
distilgpt2	NMT	10.3687	0.2276	0.2387	0.9507	0.5781	100.0000	0.2171	0.2388	0.9284	0.9722
distilgpt2	NQM	9.5684	0.2144	0.2540	0.8464	0.4061	100.0000	0.2056	0.2239	0.8249	0.8692
distilgpt2	NQT	15.8990	0.3385	0.2403	1.4202	0.8621	100.0000	0.3241	0.3537	1.3820	1.4555
distilgpt2	QMT	9.8263	0.2492	0.3179	0.7708	0.3207	100.0000	0.2320	0.2668	0.7446	0.7972
gpt2	NMT	31.1167	0.2894	0.2137	1.3348	0.7670	100.0000	0.2736	0.3065	1.2981	1.3723
gpt2	NQM	24.9467	0.2799	0.2315	1.1947	0.7302	100.0000	0.2642	0.2955	1.1675	1.2221
gpt2	NQT	16.4593	0.1562	0.1777	0.8666	0.4331	100.0000	0.1466	0.1658	0.8356	0.9004
gpt2	QMT	14.3721	0.1705	0.3167	0.5393	0.1852	100.0000	0.1621	0.1797	0.5192	0.5611

Antisymmetry sanity check

model	pair	mean_comm_norm	mean_relative_antisym_error	mean_cos_comm_ab_neg_comm_ba	n
bert-base-uncased	MT_vs_TM	4.2108	0.0000	1.0000	100.0000
bert-base-uncased	NM_vs_MN	2.0965	0.0000	1.0000	100.0000
bert-base-uncased	NQ_vs_QN	4.0297	0.0000	1.0000	100.0000
bert-base-uncased	NT_vs_TN	4.9744	0.0000	1.0000	100.0000
bert-base-uncased	QM_vs_MQ	4.4676	0.0000	1.0000	100.0000
bert-base-uncased	QT_vs_TQ	3.0670	0.0000	1.0000	100.0000
distilroberta-base	MT_vs_TM	1.3342	0.0000	1.0000	100.0000
distilroberta-base	NM_vs_MN	1.2204	0.0000	1.0000	100.0000
distilroberta-base	NQ_vs_QN	1.4504	0.0000	1.0000	100.0000
distilroberta-base	NT_vs_TN	1.3915	0.0000	1.0000	100.0000
distilroberta-base	QM_vs_MQ	1.3567	0.0000	1.0000	100.0000
distilroberta-base	QT_vs_TQ	0.6474	0.0000	1.0000	100.0000
roberta-base	MT_vs_TM	1.3729	0.0000	1.0000	100.0000
roberta-base	NM_vs_MN	1.3408	0.0000	1.0000	100.0000
roberta-base	NQ_vs_QN	1.6280	0.0000	1.0000	100.0000
roberta-base	NT_vs_TN	1.4048	0.0000	1.0000	100.0000
roberta-base	QM_vs_MQ	1.5514	0.0000	1.0000	100.0000
roberta-base	QT_vs_TQ	0.7724	0.0000	1.0000	100.0000

Semantic equivalence control

model	label	mean_noncommutativity	std_noncommutativity	mean_relative_commutator_std	std_relative_commutator_nor	mean_cosine	std_cosine	n
bert-base-uncased	equivalent	0.1130	0.0409	0.4818	0.0809	0.8870	0.0409	400.0000
bert-base-uncased	non_equivalent	0.3291	0.1134	0.8065	0.1326	0.6709	0.1134	400.0000
distilroberta-base	equivalent	0.1295	0.0483	0.5406	0.1277	0.8705	0.0483	400.0000
distilroberta-base	non_equivalent	0.2319	0.0381	0.6868	0.0560	0.7681	0.0381	400.0000
roberta-base	equivalent	0.1543	0.0630	0.5831	0.1304	0.8457	0.0630	400.0000
roberta-base	non_equivalent	0.2784	0.0439	0.7465	0.0597	0.7216	0.0439	400.0000

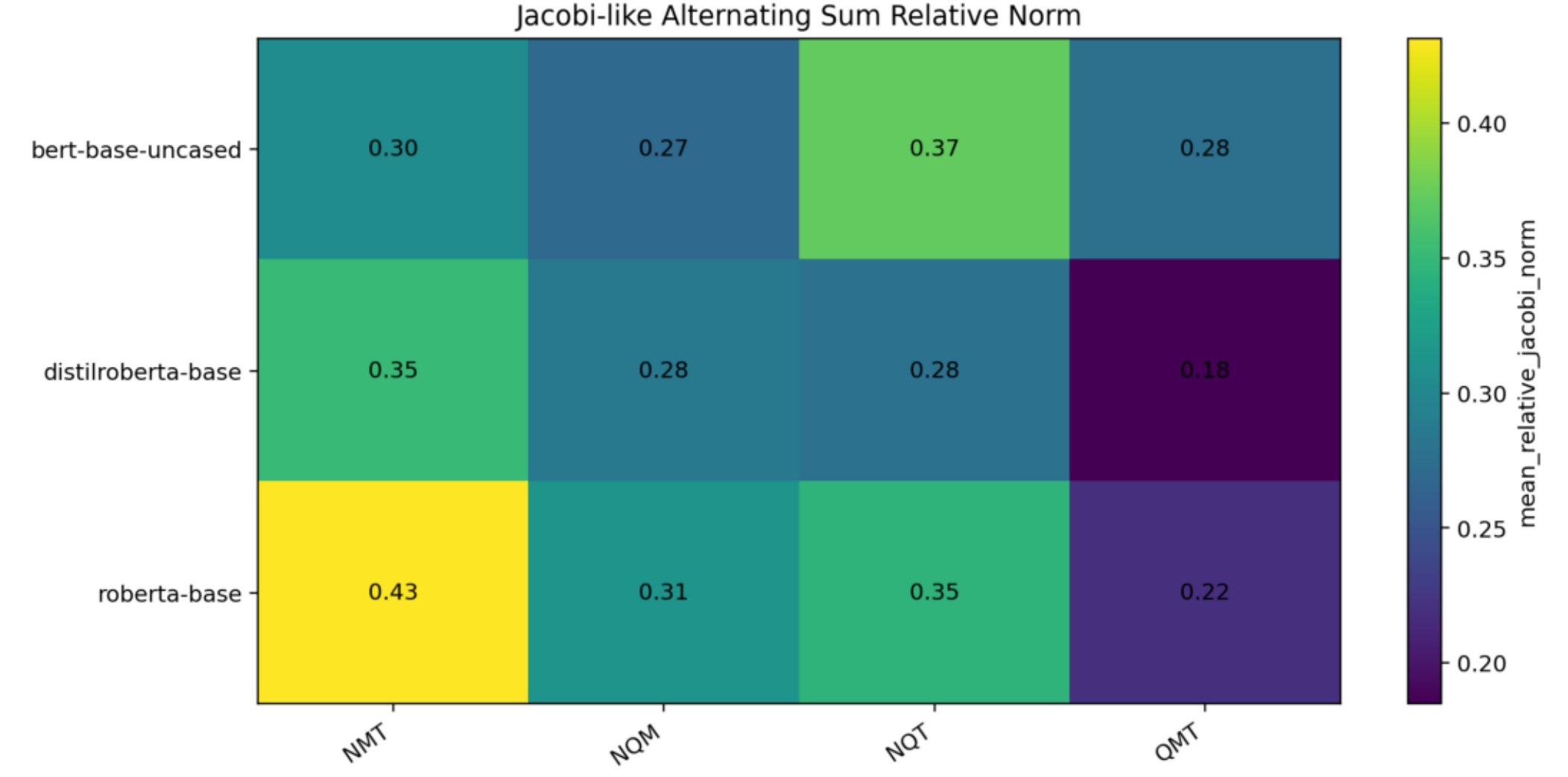
Semantic equivalence effect sizes

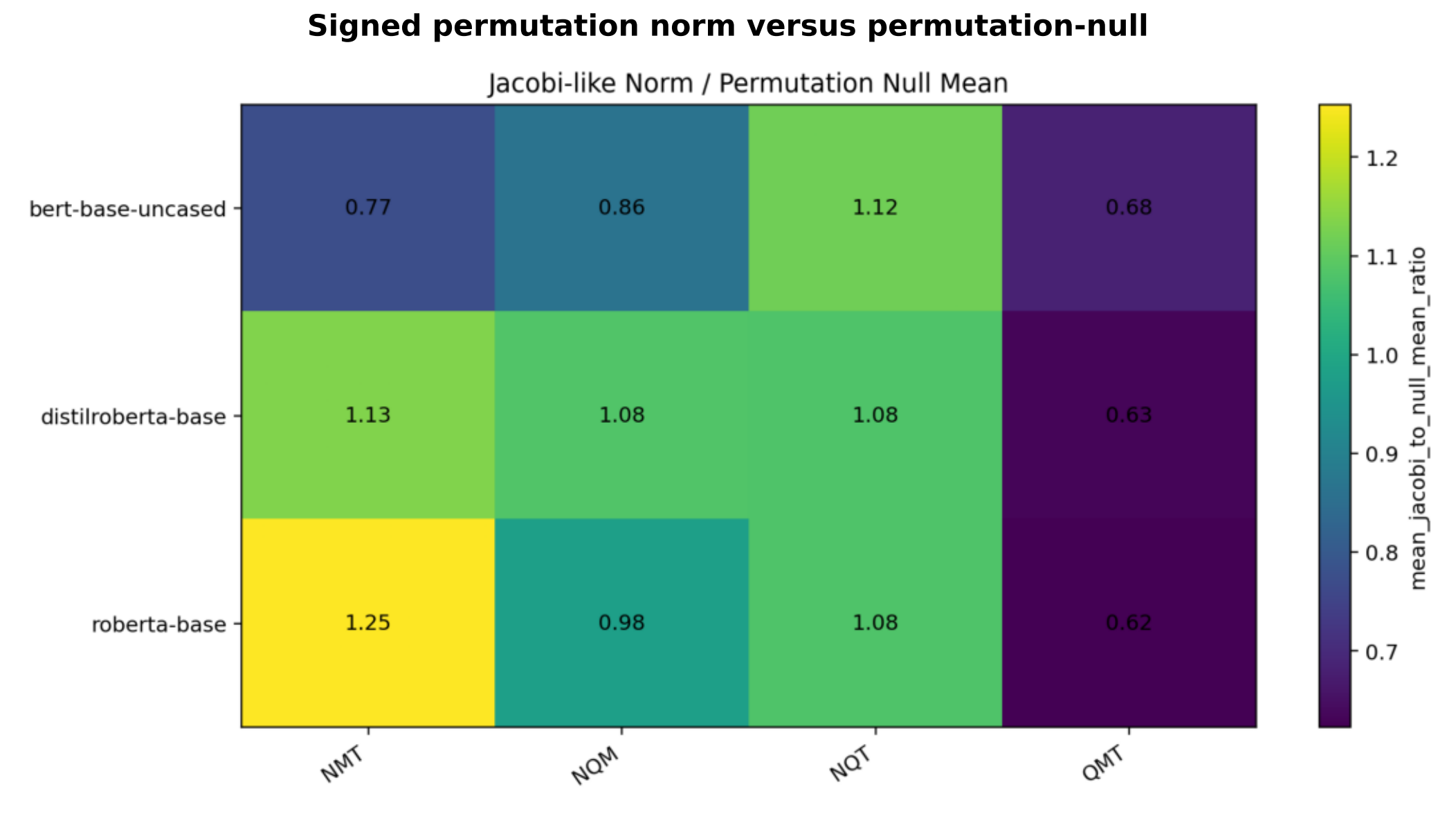
model	equiv_mean	non_equiv_mean	diff	cohens_d	equiv_std	non_equiv_std
bert-base-uncased	0.1130	0.3291	0.2161	2.5344	0.0409	0.1134
distilroberta-base	0.1295	0.2319	0.1024	2.3536	0.0483	0.0381
roberta-base	0.1543	0.2784	0.1241	2.2854	0.0630	0.0439

Composition summary: strongest noncommutativity rows

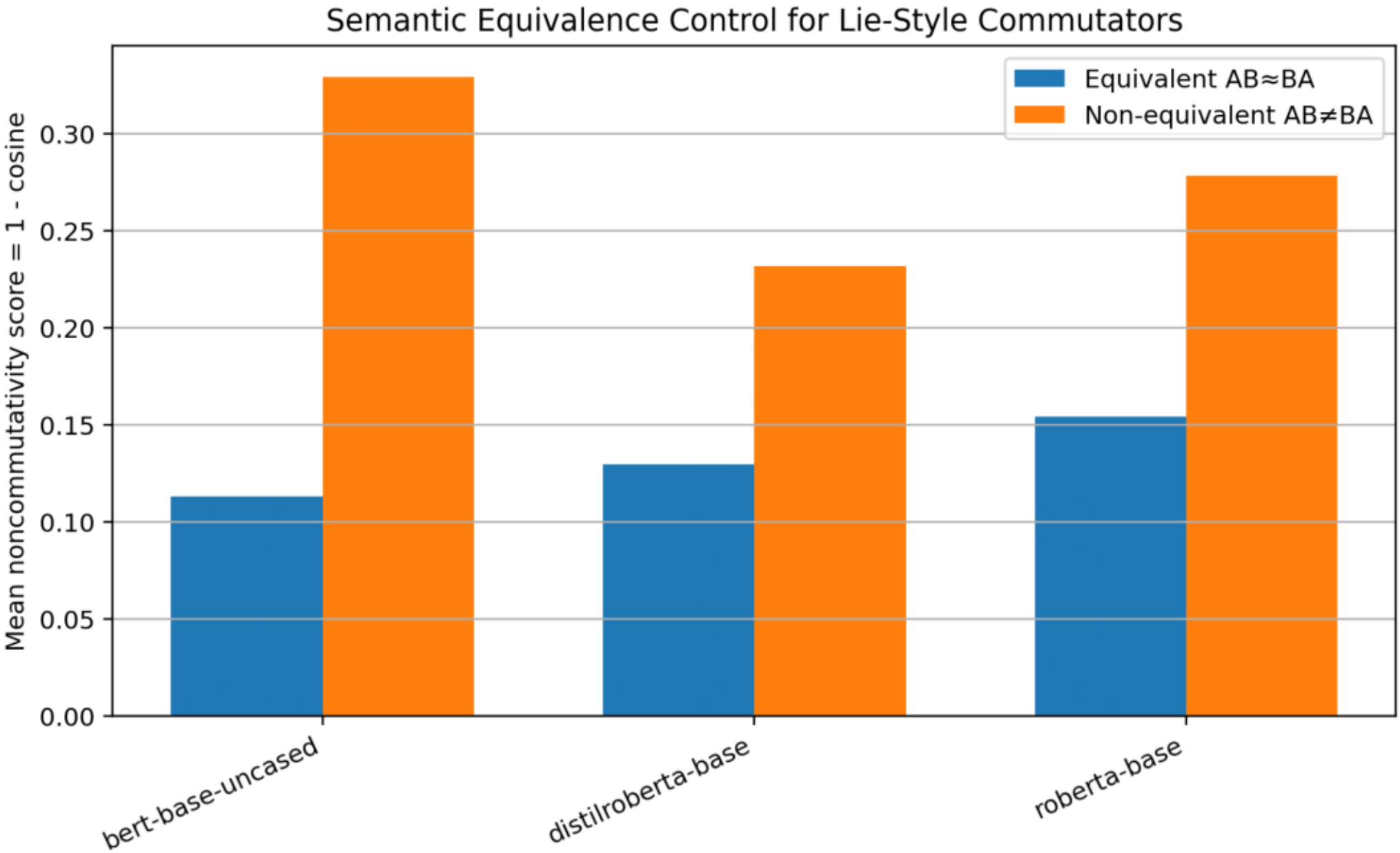
model	pair	mean_commutator_nor	std_commutator_norm	mean_relative_commutato	std_relative_commutator_r	mean_cosine	std_cosine	mean_noncommutativit	std_noncommutativity	n
bert-base-uncased	NT_vs_TN	5.1565	0.5832	0.9908	0.1066	0.5102	0.1009	0.4898	0.1009	80.0000
roberta-base	NQ_vs_QN	1.6422	0.0726	0.7846	0.0364	0.6928	0.0279	0.3072	0.0279	80.0000
bert-base-uncased	MT_vs_TM	4.2726	0.3075	0.7817	0.0954	0.6938	0.0727	0.3062	0.0727	80.0000
roberta-base	QM_vs_MQ	1.5649	0.0863	0.7714	0.0541	0.7032	0.0410	0.2968	0.0410	80.0000
distilroberta-base	MT_vs_TM	1.3504	0.0606	0.7683	0.0387	0.7124	0.0302	0.2876	0.0302	80.0000
bert-base-uncased	QM_vs_MQ	4.4793	0.2156	0.7341	0.0487	0.7340	0.0347	0.2660	0.0347	80.0000
roberta-base	MT_vs_TM	1.3956	0.0770	0.7215	0.0560	0.7411	0.0394	0.2589	0.0394	80.0000
roberta-base	NT_vs_TN	1.4185	0.1071	0.7172	0.0599	0.7429	0.0428	0.2571	0.0428	80.0000
bert-base-uncased	NQ_vs_QN	4.0903	0.2939	0.6970	0.0493	0.7598	0.0347	0.2402	0.0347	80.0000
distilroberta-base	NQ_vs_QN	1.4541	0.0644	0.6731	0.0384	0.7735	0.0257	0.2265	0.0257	80.0000
distilroberta-base	NT_vs_TN	1.3975	0.0725	0.6675	0.0342	0.7783	0.0232	0.2217	0.0232	80.0000
distilroberta-base	QM_vs_MQ	1.3672	0.0842	0.6613	0.0460	0.7926	0.0287	0.2074	0.0287	80.0000
bert-base-uncased	QT_vs_TQ	3.1001	0.1729	0.5463	0.0440	0.8513	0.0231	0.1487	0.0231	80.0000
roberta-base	QT_vs_TQ	0.7784	0.0701	0.3860	0.0342	0.9269	0.0128	0.0731	0.0128	80.0000
distilroberta-base	QT_vs_TQ	0.6580	0.0281	0.3422	0.0173	0.9415	0.0059	0.0585	0.0059	80.0000
bert-base-uncased	NM_vs_MN	1.2527	0.2368	0.2305	0.0538	0.9728	0.0125	0.0272	0.0125	80.0000
roberta-base	NM_vs_MN	0.3865	0.0403	0.2032	0.0248	0.9794	0.0050	0.0206	0.0050	80.0000
distilroberta-base	NM_vs_MN	0.3128	0.0212	0.1721	0.0201	0.9852	0.0036	0.0148	0.0036	80.0000

Signed permutation relative norm

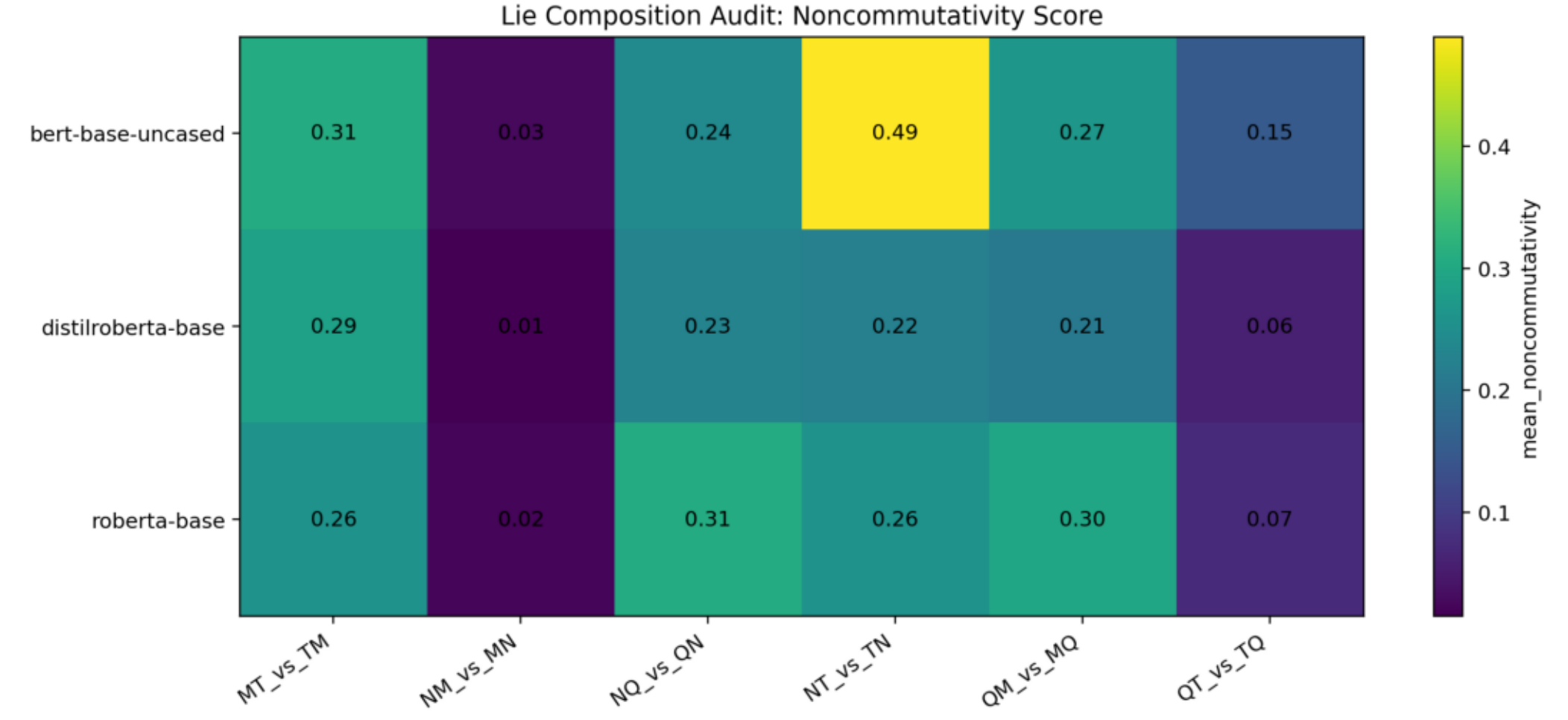



Signed permutation norm versus permutation-null
Jacobi-like Norm / Permutation Null Mean
bert-base-uncased
distilroberta-base
roberta-base
0.77
0.86
1.12
0.68
1.13
1.08
1.08
0.63
1.25
0.98
1.08
0.62
NMT
NQM
NQT
QMT
mean_jacobi_to_null_mean_ratio
1.2
1.1
1.0
0.9
0.8
0.7

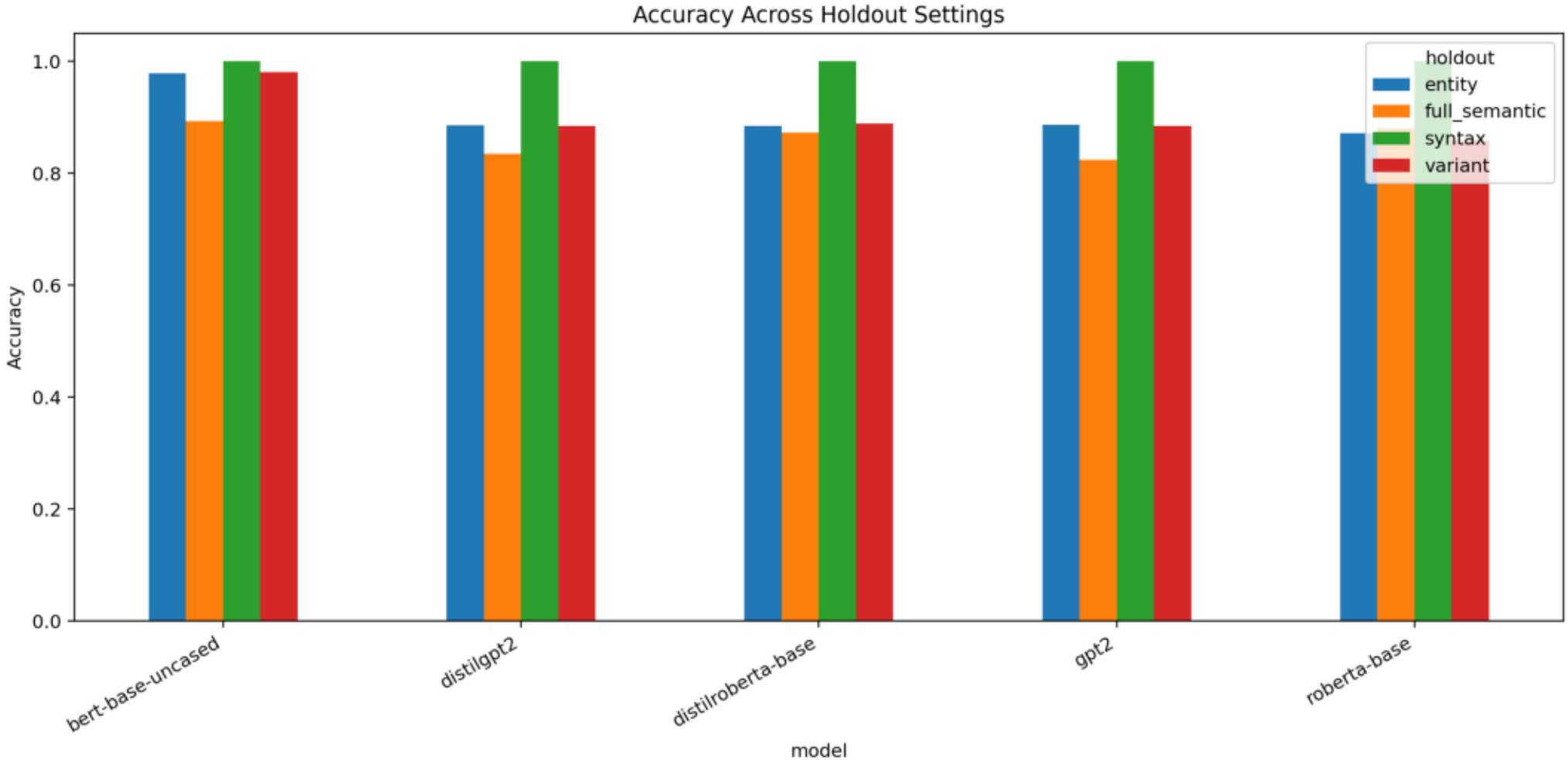
Semantic equivalent vs non-equivalent control



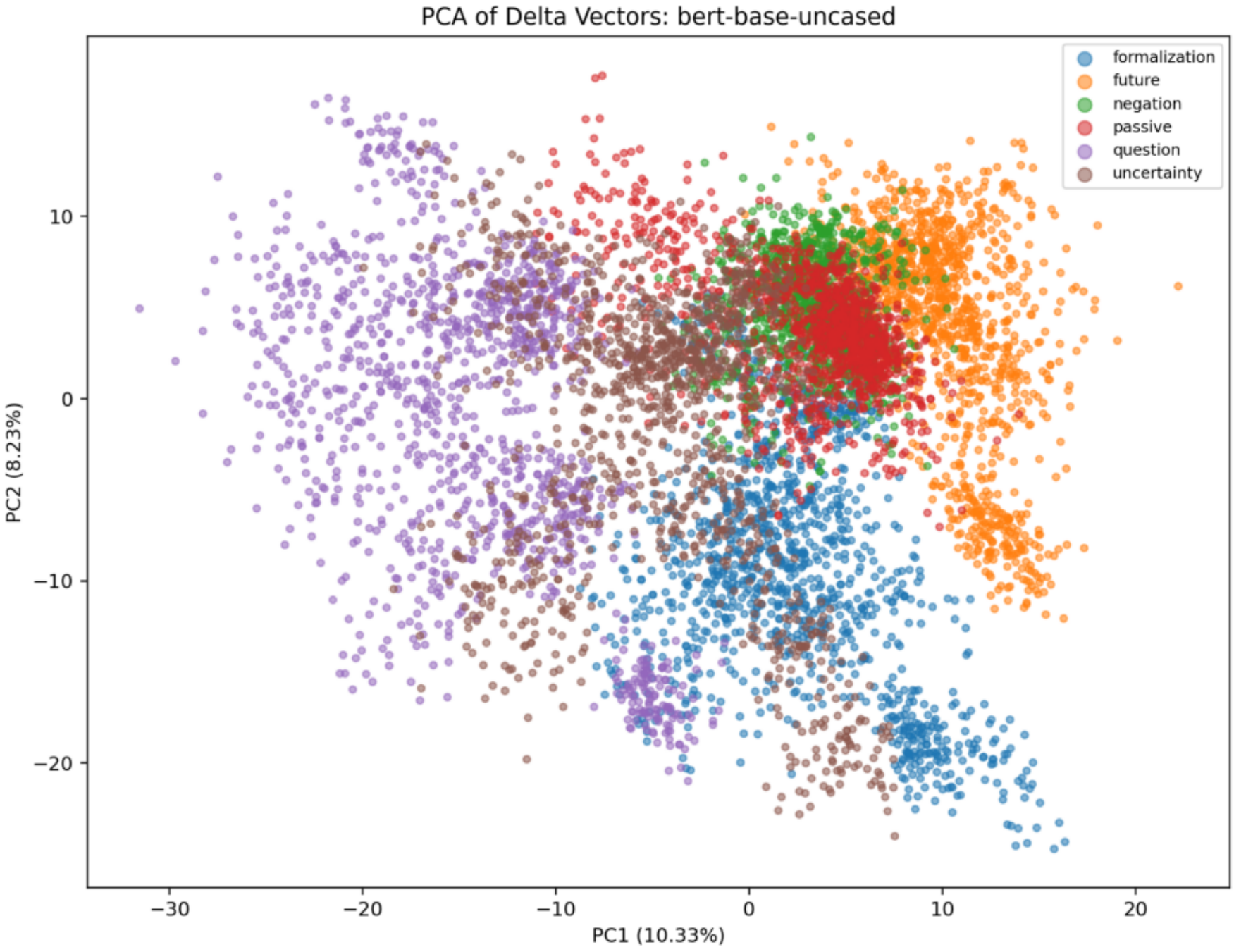
Composition noncommutativity heatmap



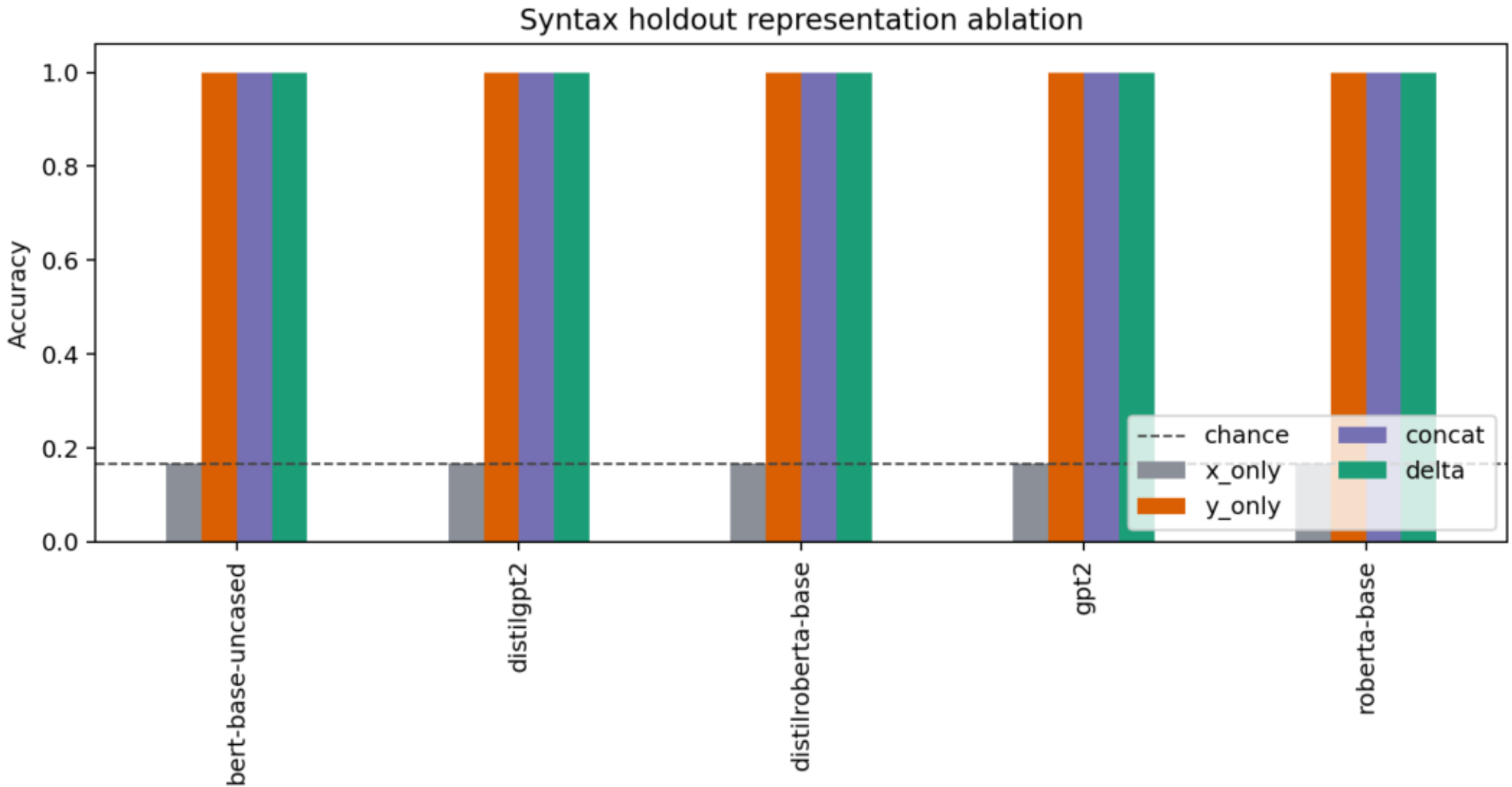
Original holdout accuracy comparison



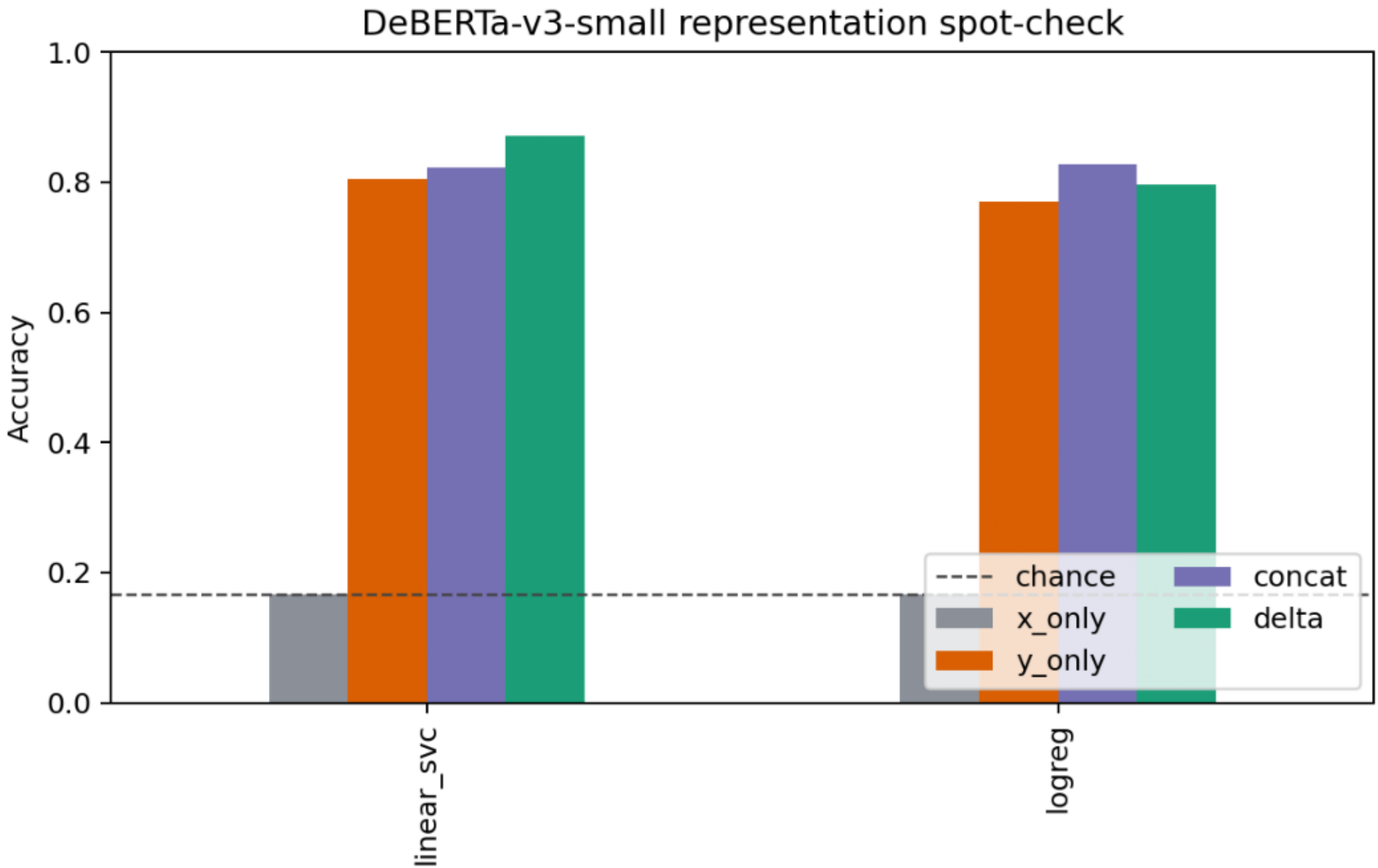
Original PCA class geometry



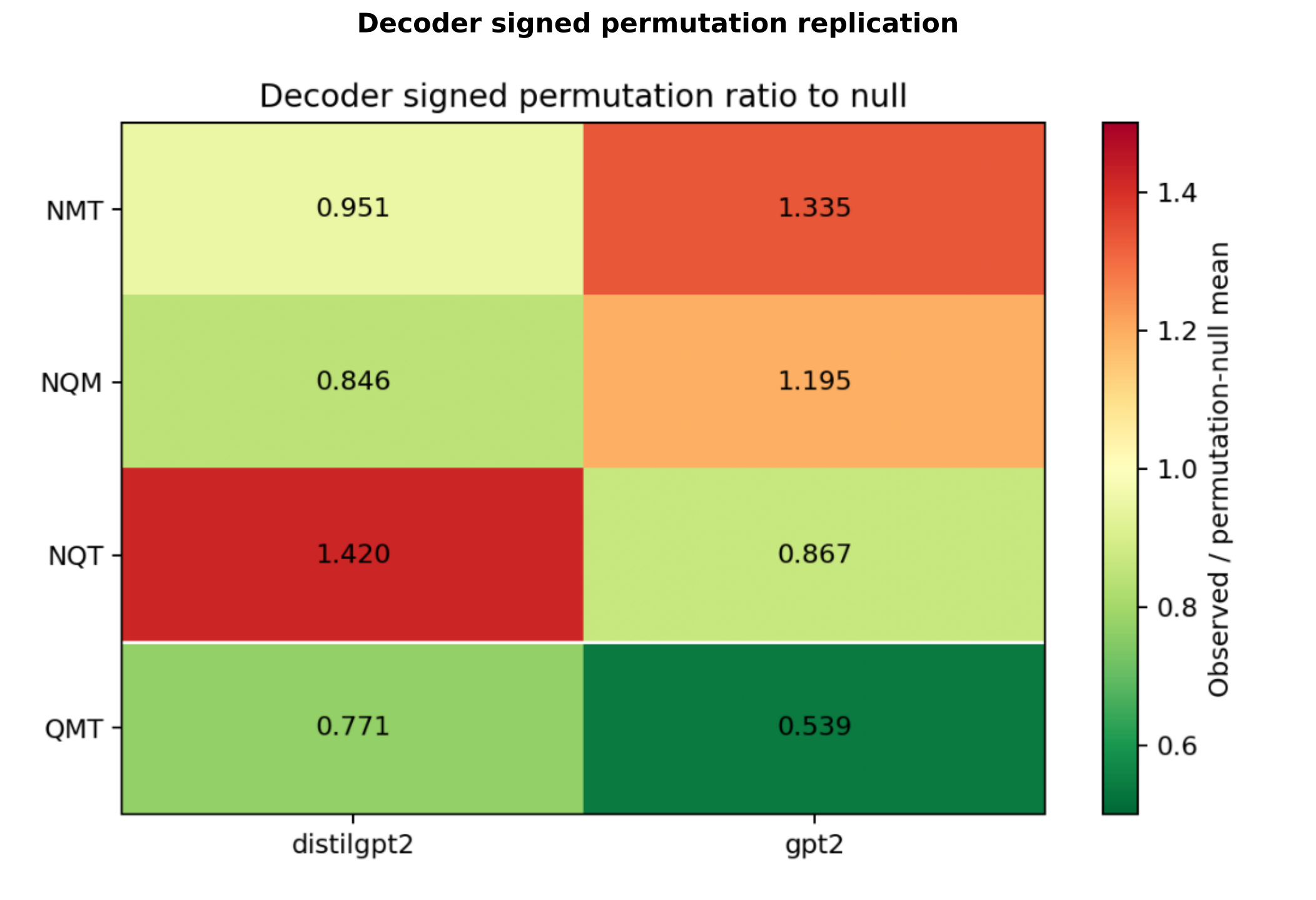
Syntax representation ablation



DeBERTa-v3-small spot-check



Decoder signed permutation replication



Audit notes for external verifier

Antisymmetry is not treated as evidence because the implementation defines [B,A] as the negative order of [A,B].

A literal nested-commutator Jacobi expression over the same six endpoint vectors cancels algebraically. The reported third-order diagnostic is the non-tautological signed permutation composition sum ABC+BCA+CAB-ACB-CBA-BAC.

Duplicate endpoint templates were detected and removed before finalizing today's result. The final dataset has 400 Jacobi rows and zero duplicate endpoint sets.

New Track 1 result: syntax y_only=1.0 confirms target/surface leakage for the syntax split. New Track 1 spot-check: DeBERTa-v3-small supports delta superiority for Linear SVC but not for logistic regression. New Track 2 result: GPT-2 and DistilGPT-2 both keep QMT below permutation null, while negation triples remain mixed.

Remaining blockers: representation-ablation tables for every non-syntax holdout split, full-semantic pooling ablation, grammar-generated templates, endpoint-only controls for Track 2, focused negation analysis, and prior-work positioning.

Code listing: run_lie_algebraic_identities.py (1)

```
0001: import gc
0002: import json
0003: import os
0004: import warnings
0005: from pathlib import Path
0006:
0007: import numpy as np
0008: import pandas as pd
0009: import torch
0010: import matplotlib.pyplot as plt
0011:
0012: from transformers import AutoTokenizer, AutoModel
0013: from sklearn.decomposition import PCA
0014: from sklearn.metrics.pairwise import cosine_similarity
0015:
0016: warnings.filterwarnings("ignore")
0017:
0018:
0019: class LieAlgebraicIdentitiesAudit:
0020:     def __init__(self):
0021:         self.out_dir = Path(os.getenv("LIE_ALGEBRA_OUT_DIR", "lie_algebraic_identities_results"))
0022:         self.csv_dir = self.out_dir / "csv"
0023:         self.fig_dir = self.out_dir / "figures"
0024:         self.csv_dir.mkdir(parents=True, exist_ok=True)
0025:         self.fig_dir.mkdir(parents=True, exist_ok=True)
0026:
0027:         self.device = "cuda" if torch.cuda.is_available() else "cpu"
0028:         self.hf_token = None
0029:         self.pca_dim = 64
0030:         self.n_jacobi_null = 1000
0031:         self.n_bootstrap = 2000
0032:
0033:         self.models = [
0034:             model.strip()
0035:             for model in os.getenv(
0036:                 "LIE_ALGEBRA_MODELS",
0037:                 "bert-base-uncased,distilroberta-base,roberta-base",
0038:             ).split(",")
0039:             if model.strip()
0040:         ]
0041:
0042:         self.ops = ["N", "Q", "M", "T"]
0043:
0044:         self.subjects = [
0045:             "scientist", "engineer", "teacher", "doctor", "programmer",
0046:             "researcher", "analyst", "manager", "wizard", "dragon",
0047:             "queen", "robot", "pirate", "oracle", "knight", "alien",
0048:         ]
0049:
0050:         self.actions = [
0051:             ("accepted", "accept", "the explanation"),
0052:             ("completed", "complete", "the repair"),
0053:             ("confirmed", "confirm", "the answer"),
0054:             ("approved", "approve", "the treatment"),
```

Code listing: run_lie_algebraic_identities.py (2)

```
            ("fixed", "fix", "the bug"),
            ("supported", "support", "the theory"),
            ("verified", "verify", "the report"),
            ("guarded", "guard", "the treasure"),
            ("opened", "open", "the portal"),
            ("signed", "sign", "the treaty"),
        ]

    def log(self, msg):
        print(msg, flush=True)

    def base_sentence(self, subject, past, obj):
        return f"The {subject} {past} {obj}."

    def compose(self, seq, subject, past, base, obj):
        s = "".join(seq)

        mapping = {
            "": f"The {subject} {past} {obj}.",

            "N": f"The {subject} failed to {base} {obj}.",
            "Q": f"Could the {subject} {base} {obj}?",
            "M": f"The {subject} allegedly {past} {obj}.",
            "T": f"The {subject} will {base} {obj} tomorrow.",

            "NQ": f"Could the {subject} fail to {base} {obj}?",
            "QN": f"Is it false that the {subject} {past} {obj}?",

            "NM": f"The {subject} allegedly failed to {base} {obj}.",
            "MN": f"Allegedly, the {subject} failed to {base} {obj}.",

            "NT": f"The {subject} will fail to {base} {obj} tomorrow.",
            "TN": f"The {subject} failed to have {past} {obj} earlier.",

            "QM": f"Could the {subject} allegedly {base} {obj}?",
            "MQ": f"Is it alleged that the {subject} {past} {obj}?",

            "QT": f"Could the {subject} {base} {obj} tomorrow?",
            "TQ": f"Will the {subject} {base} {obj} tomorrow?",

            "MT": f"The {subject} will allegedly {base} {obj} tomorrow.",
            "TM": f"The {subject} was allegedly going to {base} {obj}.",
        }

        if s in mapping:
            return mapping[s]

        # canonical triple templates
        if s == "NQM":
            return f"Could the {subject} allegedly fail to {base} {obj}?"
        if s == "QMN":
            return f"Is it false that the {subject} allegedly {past} {obj}?"
        if s == "MNQ":
            return f"Is it alleged that the {subject} failed to {base} {obj}?"
```

Code listing: run_lie_algebraic_identities.py (3)

```
0109: 
0110:         if s == "QNM":
0111:             return f"Could it be false that the {subject} allegedly {past} {obj}?"
0112:         if s == "NMQ":
0113:             return f"Could it be alleged that the {subject} failed to {base} {obj}?"
0114:         if s == "MQN":
0115:             return f"Allegedly, is it false that the {subject} {past} {obj}?"
0116: 
0117:         if s == "NMT":
0118:             return f"The {subject} will allegedly fail to {base} {obj} tomorrow."
0119:         if s == "MTN":
0120:             return f"The {subject} was allegedly going to fail to {base} {obj}."
0121:         if s == "TNM":
0122:             return f"Earlier, the {subject} allegedly failed to have {past} {obj}."
0123: 
0124:         if s == "NTM":
0125:             return f"Allegedly, the {subject} will fail to {base} {obj} tomorrow."
0126:         if s == "TMN":
0127:             return f"Earlier, it was alleged that the {subject} failed to have {past} {obj}."
0128:         if s == "MNT":
0129:             return f"It is alleged that the {subject} will fail to {base} {obj} tomorrow."
0130: 
0131:         if s == "NQT":
0132:             return f"Could the {subject} fail to {base} {obj} tomorrow?"
0133:         if s == "QTN":
0134:             return f"Is it false that the {subject} will {base} {obj} tomorrow?"
0135:         if s == "TNQ":
0136:             return f"Could the {subject} have failed to {base} {obj} earlier?"
0137: 
0138:         if s == "QNT":
0139:             return f"Could it be false that the {subject} will {base} {obj} tomorrow?"
0140:         if s == "NTQ":
0141:             return f"Could it be that the {subject} will fail to {base} {obj} tomorrow?"
0142:         if s == "TQN":
0143:             return f"Will it be false that the {subject} {base} {obj} tomorrow?"
0144: 
0145:         if s == "QMT":
0146:             return f"Could the {subject} allegedly {base} {obj} tomorrow?"
0147:         if s == "MTQ":
0148:             return f"Could it be alleged that the {subject} will {base} {obj} tomorrow?"
0149:         if s == "TQM":
0150:             return f"Allegedly, will the {subject} {base} {obj} tomorrow?"
0151: 
0152:         if s == "MQT":
0153:             return f"Will it be alleged that the {subject} {base} {obj} tomorrow?"
0154:         if s == "QTM":
0155:             return f"Allegedly, could the {subject} {base} {obj} tomorrow?"
0156:         if s == "TMQ":
0157:             return f"Could the {subject} have allegedly planned to {base} {obj}?"
0158: 
0159:         raise ValueError(f"Unsupported composition: {s}")
0160: 
0161:     def build_dataset(self, n_templates=100, seed=42):
0162:         rng = np.random.default_rng(seed)
```

Code listing: run_lie_algebraic_identities.py (4)

```
        combos = [(s, a) for s in self.subjects for a in self.actions]
        rng.shuffle(combos)
        combos = combos[:n_templates]

        rows = []

        pairs = [("N", "Q"), ("N", "M"), ("N", "T"), ("Q", "M"), ("Q", "T"), ("M", "T")]
        triples = [("N", "Q", "M"), ("N", "Q", "T"), ("N", "M", "T"), ("Q", "M", "T")]

        for i, (subject, action) in enumerate(combos):
            past, base, obj = action
            source = self.base_sentence(subject, past, obj)

            for a, b in pairs:
                rows.append({
                    "template_id": i,
                    "kind": "antisymmetry",
                    "source": source,
                    "a": a,
                    "b": b,
                    "c": "",
                    "ab": self.compose([a, b], subject, past, base, obj),
                    "ba": self.compose([b, a], subject, past, base, obj),
                })

            for a, b, c in triples:
                rows.append({
                    "template_id": i,
                    "kind": "jacobi",
                    "source": source,
                    "a": a,
                    "b": b,
                    "c": c,
                    "abc": self.compose([a, b, c], subject, past, base, obj),
                    "bca": self.compose([b, c, a], subject, past, base, obj),
                    "cab": self.compose([c, a, b], subject, past, base, obj),
                    "acb": self.compose([a, c, b], subject, past, base, obj),
                    "cba": self.compose([c, b, a], subject, past, base, obj),
                    "bac": self.compose([b, a, c], subject, past, base, obj),
                })

        df = pd.DataFrame(rows)
        df.to_csv(self.csv_dir / "lie_algebraic_identities_dataset.csv", index=False)
        return df

    @torch.no_grad()
    def get_embeddings(self, model_name, texts, batch_size=16):
        tokenizer = AutoTokenizer.from_pretrained(model_name, token=self.hf_token)

        if tokenizer.pad_token is None:
            tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"

        model = AutoModel.from_pretrained(model_name, token=self.hf_token).to(self.device)
        model.eval()
```

Code listing: run_lie_algebraic_identities.py (5)

```
0217:
0218:         embs = []
0219:
0220:         for i in range(0, len(texts), batch_size):
0221:             batch = texts[i:i + batch_size]
0222:
0223:             enc = tokenizer(
0224:                 batch,
0225:                 padding=True,
0226:                 truncation=True,
0227:                 max_length=128,
0228:                 return_tensors="pt",
0229:             ).to(self.device)
0230:
0231:             out = model(**enc)
0232:             hidden = out.last_hidden_state
0233:             mask = enc["attention_mask"].unsqueeze(-1)
0234:
0235:             pooled = (hidden * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
0236:             embs.append(pooled.detach().cpu().numpy())
0237:
0238:         del model
0239:         gc.collect()
0240:
0241:         if self.device == "cuda":
0242:             torch.cuda.empty_cache()
0243:
0244:         return np.vstack(embs)
0245:
0246:     def cos(self, x, y):
0247:         return float(cosine_similarity(x.reshape(1, -1), y.reshape(1, -1))[0, 0])
0248:
0249:     def alternating_null_stats(self, vectors, observed_norm, scale, rng):
0250:         null_relative = []
0251:         n = len(vectors)
0252:
0253:         for _ in range(self.n_jacobi_null):
0254:             positive = set(rng.choice(n, size=n // 2, replace=False).tolist())
0255:             signed_sum = np.zeros_like(vectors[0])
0256:
0257:             for i, vector in enumerate(vectors):
0258:                 signed_sum += vector if i in positive else -vector
0259:
0260:             null_relative.append(np.linalg.norm(signed_sum) / scale)
0261:
0262:         null_relative = np.asarray(null_relative, dtype=float)
0263:         observed_relative = observed_norm / scale
0264:
0265:         return {
0266:             "null_mean_relative_jacobi_norm": float(null_relative.mean()),
0267:             "null_std_relative_jacobi_norm": float(null_relative.std(ddof=1)),
0268:             "jacobi_to_null_mean_ratio": float(observed_relative / (null_relative.mean() + 1e-12)),
0269:             "null_percentile_smaller_is_better": float((null_relative <= observed_relative).mean()),
0270:         }
```

Code listing: run_lie_algebraic_identities.py (6)

```
0271: 
0272:     def bootstrap_mean_ci(self, values, rng, alpha=0.05):
0273:         values = np.asarray(values, dtype=float)
0274:         if len(values) == 0:
0275:             return np.nan, np.nan
0276: 
0277:         draws = rng.choice(values, size=(self.n_bootstrap, len(values)), replace=True).mean(axis=1)
0278:         return (
0279:             float(np.quantile(draws, alpha / 2)),
0280:             float(np.quantile(draws, 1 - alpha / 2)),
0281:         )
0282: 
0283:     def compute_for_model(self, model_name, df):
0284:         self.log("\n" + "=" * 80)
0285:         self.log(f"MODEL: {model_name}")
0286:         self.log("=" * 80)
0287: 
0288:         text_cols = ["source", "ab", "ba", "abc", "bca", "cab", "acb", "cba", "bac"]
0289:         all_texts = set()
0290: 
0291:         for col in text_cols:
0292:             if col in df.columns:
0293:                 all_texts |= set(df[col].dropna().values)
0294: 
0295:         all_texts = sorted(all_texts)
0296:         idx = {t: i for i, t in enumerate(all_texts)}
0297: 
0298:         raw = self.get_embeddings(model_name, all_texts)
0299: 
0300:         dim = min(self.pca_dim, raw.shape[0] - 1, raw.shape[1])
0301:         pca = PCA(n_components=dim, random_state=42)
0302:         vecs = pca.fit_transform(raw)
0303: 
0304:         anti_rows = []
0305:         jacobi_rows = []
0306: 
0307:         anti_df = df[df["kind"] == "antisymmetry"]
0308: 
0309:         for row in anti_df.itertuples():
0310:             x = vecs[idx[row.source]]
0311:             ab = vecs[idx[row.ab]]
0312:             ba = vecs[idx[row.ba]]
0313: 
0314:             delta_ab = ab - x
0315:             delta_ba = ba - x
0316: 
0317:             comm_ab = delta_ab - delta_ba
0318:             comm_ba = delta_ba - delta_ab
0319: 
0320:             antisym_error = np.linalg.norm(comm_ab + comm_ba)
0321:             comm_norm = np.linalg.norm(comm_ab)
0322: 
0323:             anti_rows.append({
0324:                 "model": model_name,
```

Code listing: run_lie_algebraic_identities.py (7)

```
                "template_id": row.template_id,
                "pair": f"{row.a}{row.b}_vs_{row.b}{row.a}",
                "a": row.a,
                "b": row.b,
                "comm_norm": float(comm_norm),
                "antisym_error": float(antisym_error),
                "relative_antisym_error": float(antisym_error / (comm_norm + 1e-12)),
                "cos_comm_ab_neg_comm_ba": self.cos(comm_ab, -comm_ba),
            })

        jacobi_df = df[df["kind"] == "jacobi"]
        seed = sum((i + 1) * ord(ch) for i, ch in enumerate(model_name)) % (2 ** 32)
        rng = np.random.default_rng(seed)

        for row in jacobi_df.itertuples():
            x = vecs[idx[row.source]]

            abc = vecs[idx[row.abc]] - x
            bca = vecs[idx[row.bca]] - x
            cab = vecs[idx[row.cab]] - x

            acb = vecs[idx[row.acb]] - x
            cba = vecs[idx[row.cba]] - x
            bac = vecs[idx[row.bac]] - x

            # Jacobi-like alternating sum over permutations:
            # J = ABC + BCA + CAB - ACB - CBA - BAC
            jacobi = abc + bca + cab - acb - cba - bac

            positive_norm = np.linalg.norm(abc) + np.linalg.norm(bca) + np.linalg.norm(cab)
            negative_norm = np.linalg.norm(acb) + np.linalg.norm(cba) + np.linalg.norm(bac)
            scale = 0.5 * (positive_norm + negative_norm) + 1e-12

            jacobi_norm = np.linalg.norm(jacobi)
            null_stats = self.alternating_null_stats(
                [abc, bca, cab, acb, cba, bac],
                jacobi_norm,
                scale,
                rng,
            )

            jacobi_rows.append({
                "model": model_name,
                "template_id": row.template_id,
                "triple": f"{row.a}{row.b}{row.c}",
                "a": row.a,
                "b": row.b,
                "c": row.c,
                "jacobi_norm": float(jacobi_norm),
                "relative_jacobi_norm": float(jacobi_norm / scale),
                **null_stats,
            })

        anti_result = pd.DataFrame(anti_rows)
```

Code listing: run_lie_algebraic_identities.py (8)

```
        jacobi_result = pd.DataFrame(jacobi_rows)

        anti_result.to_csv(self.csv_dir / f"antisymmetry_raw_{self.safe_name(model_name)}.csv", index=False)
        jacobi_result.to_csv(self.csv_dir / f"jacobi_raw_{self.safe_name(model_name)}.csv", index=False)

        return anti_result, jacobi_result

    def safe_name(self, model_name):
        return model_name.replace("/", "_").replace("-", "_")

    def summarize(self, anti_all, jacobi_all):
        anti_summary = (
            anti_all.groupby(["model", "pair"])
            .agg(
                mean_comm_norm=("comm_norm", "mean"),
                mean_relative_antisym_error=("relative_antisym_error", "mean"),
                mean_cos_comm_ab_neg_comm_ba=("cos_comm_ab_neg_comm_ba", "mean"),
                n=("pair", "count"),
            )
            .reset_index()
        )

        jacobi_summary = (
            jacobi_all.groupby(["model", "triple"])
            .agg(
                mean_jacobi_norm=("jacobi_norm", "mean"),
                mean_relative_jacobi_norm=("relative_jacobi_norm", "mean"),
                mean_null_relative_jacobi_norm=("null_mean_relative_jacobi_norm", "mean"),
                mean_jacobi_to_null_mean_ratio=("jacobi_to_null_mean_ratio", "mean"),
                mean_null_percentile_smaller_is_better=("null_percentile_smaller_is_better", "mean"),
                n=("triple", "count"),
            )
            .reset_index()
        )

        rng = np.random.default_rng(20260611)
        ci_rows = []

        for row in jacobi_summary.itertuples():
            group = jacobi_all[
                (jacobi_all["model"] == row.model)
                & (jacobi_all["triple"] == row.triple)
            ]

            rel_low, rel_high = self.bootstrap_mean_ci(group["relative_jacobi_norm"].values, rng)
            ratio_low, ratio_high = self.bootstrap_mean_ci(group["jacobi_to_null_mean_ratio"].values, rng)

            ci_rows.append({
                "model": row.model,
                "triple": row.triple,
                "relative_jacobi_norm_ci95_low": rel_low,
                "relative_jacobi_norm_ci95_high": rel_high,
                "jacobi_to_null_mean_ratio_ci95_low": ratio_low,
                "jacobi_to_null_mean_ratio_ci95_high": ratio_high,
```

Code listing: run_lie_algebraic_identities.py (9)

```
            })

        jacobi_summary = jacobi_summary.merge(pd.DataFrame(ci_rows), on=["model", "triple"], how="left")

        anti_summary.to_csv(self.csv_dir / "antisymmetry_summary.csv", index=False)
        jacobi_summary.to_csv(self.csv_dir / "jacobi_summary.csv", index=False)

        return anti_summary, jacobi_summary

    def plot_heatmap(self, df, index_col, value_col, filename, title):
        rows = sorted(df["model"].unique())
        cols = sorted(df[index_col].unique())

        mat = np.zeros((len(rows), len(cols)))

        for i, model in enumerate(rows):
            for j, col in enumerate(cols):
                val = df[(df["model"] == model) & (df[index_col] == col)][value_col].values
                mat[i, j] = val[0] if len(val) else np.nan

        plt.figure(figsize=(10, 5))
        plt.imshow(mat, aspect="auto")
        plt.colorbar(label=value_col)

        plt.xticks(np.arange(len(cols)), cols, rotation=35, ha="right")
        plt.yticks(np.arange(len(rows)), rows)
        plt.title(title)

        for i in range(mat.shape[0]):
            for j in range(mat.shape[1]):
                plt.text(j, i, f"{mat[i, j]:.2f}", ha="center", va="center")

        path = self.fig_dir / filename
        plt.tight_layout()
        plt.savefig(path, dpi=220, bbox_inches="tight")
        plt.close()
        self.log(f"Saved figure: {path}")

    def run(self):
        self.log(f"DEVICE: {self.device}")
        self.log(f"OUT DIR: {self.out_dir}")

        df = self.build_dataset(n_templates=100, seed=42)

        anti_results = []
        jacobi_results = []

        for model in self.models:
            anti, jacobi = self.compute_for_model(model, df)
            anti_results.append(anti)
            jacobi_results.append(jacobi)

        anti_all = pd.concat(anti_results, ignore_index=True)
        jacobi_all = pd.concat(jacobi_results, ignore_index=True)
```

Code listing: run_lie_algebraic_identities.py (10)

```
0487:
0488:         anti_all.to_csv(self.csv_dir / "antisymmetry_raw_all_models.csv", index=False)
0489:         jacobi_all.to_csv(self.csv_dir / "jacobi_raw_all_models.csv", index=False)
0490:
0491:         anti_summary, jacobi_summary = self.summarize(anti_all, jacobi_all)
0492:
0493:         self.plot_heatmap(
0494:             anti_summary,
0495:             "pair",
0496:             "mean_cos_comm_ab_neg_comm_ba",
0497:             "01_antisymmetry_cosine_heatmap.png",
0498:             "Antisymmetry Check: cos([A,B], -[B,A])",
0499:         )
0500:
0501:         self.plot_heatmap(
0502:             anti_summary,
0503:             "pair",
0504:             "mean_relative_antisym_error",
0505:             "02_antisymmetry_error_heatmap.png",
0506:             "Antisymmetry Error",
0507:         )
0508:
0509:         self.plot_heatmap(
0510:             jacobi_summary,
0511:             "triple",
0512:             "mean_relative_jacobi_norm",
0513:             "03_jacobi_relative_norm_heatmap.png",
0514:             "Jacobi-like Alternating Sum Relative Norm",
0515:         )
0516:
0517:         self.plot_heatmap(
0518:             jacobi_summary,
0519:             "triple",
0520:             "mean_jacobi_to_null_mean_ratio",
0521:             "04_jacobi_vs_permutation_null_heatmap.png",
0522:             "Jacobi-like Norm / Permutation Null Mean",
0523:         )
0524:
0525:         metadata = {
0526:             "antisymmetry": "checks whether [A,B] ≈ -[B,A]",
0527:             "jacobi_like": "checks alternating sum ABC+BCA+CAB-ACB-CBA-BAC",
0528:             "jacobi_null": (
0529:                 "compares the observed alternating sum with random 3-positive/3-negative "
0530:                 "sign assignments over the same six composition vectors; lower ratio and "
0531:                 "lower percentile mean stronger Jacobi-like cancellation"
0532:             ),
0533:             "caution": (
0534:                 "A literal nested-commutator Jacobi expansion over the same six embedded "
0535:                 "composition endpoints cancels by construction, so this script reports a "
0536:                 "non-tautological alternating third-order cancellation diagnostic instead."
0537:             ),
0538:             "note": "This is an empirical Lie-style diagnostic, not a formal proof of Lie algebra.",
0539:             "models": self.models,
0540:             "n_jacobi_null": self.n_jacobi_null,
```

Code listing: run_lie_algebraic_identities.py (11)

```
            "n_bootstrap": self.n_bootstrap,
            "jacobi_triples": ["NQM", "NQT", "NMT", "QMT"],
        }

        with open(self.out_dir / "metadata.json", "w", encoding="utf-8") as f:
            json.dump(metadata, f, indent=2)

        self.log("\nANTISYMMETRY SUMMARY")
        self.log(str(anti_summary))

        self.log("\nJACOBI SUMMARY")
        self.log(str(jacobi_summary))

        self.log("\nDONE")


if __name__ == "__main__":
    audit = LieAlgebraicIdentitiesAudit()
    audit.run()
```

Code listing: run_syntax_representation_ablation.py (1)

```
0001: import gc
0002: import os
0003: from pathlib import Path
0004:
0005: import numpy as np
0006: import pandas as pd
0007:
0008: from sklearn.linear_model import LogisticRegression
0009: from sklearn.metrics import accuracy_score
0010: from sklearn.pipeline import make_pipeline
0011: from sklearn.preprocessing import StandardScaler
0012: from sklearn.svm import LinearSVC
0013:
0014: import lie_llm_syntax_holdout_experiment as syntax
0015:
0016:
0017: OUT_DIR = Path(os.getenv("SYNTAX_ABLATION_OUT_DIR", "syntax_representation_ablation_results"))
0018: OUT_DIR.mkdir(parents=True, exist_ok=True)
0019:
0020: MODELS = [m.strip() for m in os.getenv(
0021:     "SYNTAX_ABLATION_MODELS",
0022:     "bert-base-uncased,roberta-base,distilroberta-base,gpt2,distilgpt2",
0023: ).split(",") if m.strip()]
0024:
0025: N_BASE = int(os.getenv("SYNTAX_ABLATION_N_BASE", str(syntax.N_BASE)))
0026:
0027:
0028: def run_classifiers(X, labels, train_mask, test_mask):
0029:     classifiers = {
0030:         "logreg": make_pipeline(
0031:             StandardScaler(),
0032:             LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1),
0033:         ),
0034:         "linear_svc": make_pipeline(
0035:             StandardScaler(),
0036:             LinearSVC(class_weight="balanced", max_iter=20000),
0037:         ),
0038:     }
0039:
0040:     rows = []
0041:     for clf_name, clf in classifiers.items():
0042:         clf.fit(X[train_mask], labels[train_mask])
0043:         pred = clf.predict(X[test_mask])
0044:         rows.append({
0045:             "classifier": clf_name,
0046:             "accuracy": float(accuracy_score(labels[test_mask], pred)),
0047:             "n_train": int(train_mask.sum()),
0048:             "n_test": int(test_mask.sum()),
0049:         })
0050:
0051:     return rows
0052:
0053:
0054: def main():
```

Code listing: run_syntax_representation_ablation.py (2)

```
    syntax.OUT_DIR = str(OUT_DIR)
    os.makedirs(syntax.OUT_DIR, exist_ok=True)

    base_df = syntax.make_base_rows(N_BASE)
    pair_df = syntax.build_pairs(base_df)
    prompts, pair_df = syntax.index_prompts(pair_df)

    base_df.to_csv(OUT_DIR / "base_sentences.csv", index=False)
    pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
    pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(
        OUT_DIR / "prompts.csv",
        index=False,
    )

    labels = pair_df["class"].astype(str).to_numpy()
    syntax_family = pair_df["syntax_family"].astype(str).to_numpy()
    train_mask = np.isin(syntax_family, ["simple_svo", "with_context"])
    test_mask = np.isin(syntax_family, ["relative_clause", "temporal_clause", "reported_statement", "conditional"])
    src_idx = pair_df["source_idx"].to_numpy()
    tgt_idx = pair_df["target_idx"].to_numpy()

    all_rows = []

    for model_name in MODELS:
        print(f"\n=== MODEL: {model_name} ===")
        vectors = syntax.get_vectors(model_name, prompts)

        x_src = vectors[src_idx]
        x_tgt = vectors[tgt_idx]
        representations = {
            "x_only": x_src,
            "y_only": x_tgt,
            "delta": x_tgt - x_src,
            "concat": np.concatenate([x_src, x_tgt], axis=1),
        }

        for rep_name, X in representations.items():
            for row in run_classifiers(X, labels, train_mask, test_mask):
                row.update({
                    "model": model_name,
                    "representation": rep_name,
                    "random_baseline": 1 / len(set(labels)),
                    "n_base": N_BASE,
                })
                print(model_name, rep_name, row["classifier"], f"{row['accuracy']:.4f}")
                all_rows.append(row)

        del vectors, x_src, x_tgt, representations
        gc.collect()

    result = pd.DataFrame(all_rows)
    result.to_csv(OUT_DIR / "syntax_representation_ablation.csv", index=False)

    pivot = result.pivot_table(
```

Code listing: run_syntax_representation_ablation.py (3)

```
0109:         index=["model", "classifier"],
0110:         columns="representation",
0111:         values="accuracy",
0112:     ).reset_index()
0113:     pivot.to_csv(OUT_DIR / "syntax_representation_ablation_pivot.csv", index=False)
0114: 
0115:     print("\nSUMMARY")
0116:     print(pivot)
0117:     print("Saved to", OUT_DIR)
0118: 
0119: 
0120: if __name__ == "__main__":
0121:     main()
```

Code listing: run_layerwise_pooling_ablation.py (1)

```
0001: import gc
0002: import os
0003: import time
0004: from pathlib import Path
0005:
0006: import numpy as np
0007: import pandas as pd
0008: import torch
0009: from sklearn.linear_model import LogisticRegression
0010: from sklearn.metrics import accuracy_score
0011: from sklearn.pipeline import make_pipeline
0012: from sklearn.preprocessing import StandardScaler
0013: from sklearn.svm import LinearSVC
0014: from transformers import AutoModel, AutoTokenizer
0015:
0016: import lie_llm_syntax_holdout_experiment as syntax
0017:
0018:
0019: OUT_DIR = Path(os.getenv("LAYERWISE_OUT_DIR", "layerwise_pooling_ablation_results"))
0020: OUT_DIR.mkdir(parents=True, exist_ok=True)
0021:
0022: MODEL_NAME = os.getenv("LAYERWISE_MODEL", "bert-base-uncased")
0023: N_BASE = int(os.getenv("LAYERWISE_N_BASE", "300"))
0024: BATCH_SIZE = int(os.getenv("LAYERWISE_BATCH_SIZE", "16"))
0025:
0026:
0027: def pool(hidden, mask, mode):
0028:     if mode == "mean":
0029:         m = mask.unsqueeze(-1).float()
0030:         return (hidden * m).sum(dim=1) / m.sum(dim=1).clamp_min(1e-9)
0031:     if mode == "cls":
0032:         return hidden[:, 0, :]
0033:     if mode == "last_token":
0034:         idx = mask.sum(dim=1).clamp_min(1) - 1
0035:         return hidden[torch.arange(hidden.shape[0]), idx]
0036:     raise ValueError(mode)
0037:
0038:
0039: def extract_layer_vectors(prompts):
0040:     tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
0041:     if tokenizer.pad_token is None:
0042:         tokenizer.pad_token = tokenizer.eos_token or tokenizer.unk_token or "[PAD]"
0043:
0044:     model = AutoModel.from_pretrained(MODEL_NAME, output_hidden_states=True)
0045:     model.eval()
0046:
0047:     rows_by_key = {}
0048:     started = time.time()
0049:
0050:     for start in range(0, len(prompts), BATCH_SIZE):
0051:         batch = prompts[start:start + BATCH_SIZE]
0052:         enc = tokenizer(batch, padding=True, truncation=True, max_length=160, return_tensors="pt")
0053:
0054:         with torch.no_grad():
```

Code listing: run_layerwise_pooling_ablation.py (2)

```
            out = model(**enc)

        for layer_idx, hidden in enumerate(out.hidden_states):
            for pooling in ["mean", "cls", "last_token"]:
                key = (layer_idx, pooling)
                rows_by_key.setdefault(key, []).append(pool(hidden, enc["attention_mask"], pooling).cpu().numpy())

        done = start + len(batch)
        print(f"{MODEL_NAME}: {done}/{len(prompts)}, {time.time() - started:.1f}s")

    result = {key: np.vstack(parts) for key, parts in rows_by_key.items()}

    del model
    gc.collect()
    return result


def fit_score(X, labels, train_mask, test_mask):
    classifiers = {
        "logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1)),
        "linear_svc": make_pipeline(StandardScaler(), LinearSVC(class_weight="balanced", max_iter=20000)),
    }

    rows = []
    for clf_name, clf in classifiers.items():
        clf.fit(X[train_mask], labels[train_mask])
        pred = clf.predict(X[test_mask])
        rows.append({
            "classifier": clf_name,
            "accuracy": float(accuracy_score(labels[test_mask], pred)),
        })
    return rows


def main():
    base_df = syntax.make_base_rows(N_BASE)
    pair_df = syntax.build_pairs(base_df)
    prompts, pair_df = syntax.index_prompts(pair_df)

    pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
    pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)

    labels = pair_df["class"].astype(str).to_numpy()
    syntax_family = pair_df["syntax_family"].astype(str).to_numpy()
    train_mask = np.isin(syntax_family, ["simple_svo", "with_context"])
    test_mask = np.isin(syntax_family, ["relative_clause", "temporal_clause", "reported_statement", "conditional"])
    src_idx = pair_df["source_idx"].to_numpy()
    tgt_idx = pair_df["target_idx"].to_numpy()

    vectors_by_key = extract_layer_vectors(prompts)

    all_rows = []
    for (layer_idx, pooling), vectors in vectors_by_key.items():
        x_src = vectors[src_idx]
```

Code listing: run_layerwise_pooling_ablation.py (3)

```
        x_tgt = vectors[tgt_idx]

        for rep_name, X in {
            "delta": x_tgt - x_src,
            "y_only": x_tgt,
        }.items():
            for row in fit_score(X, labels, train_mask, test_mask):
                row.update({
                    "model": MODEL_NAME,
                    "layer": layer_idx,
                    "pooling": pooling,
                    "representation": rep_name,
                    "n_base": N_BASE,
                    "n_train": int(train_mask.sum()),
                    "n_test": int(test_mask.sum()),
                })
                print(row)
                all_rows.append(row)

    result = pd.DataFrame(all_rows)
    result.to_csv(OUT_DIR / "layerwise_pooling_ablation.csv", index=False)

    best = (
        result[result["classifier"] == "linear_svc"]
        .sort_values("accuracy", ascending=False)
        .head(20)
    )
    best.to_csv(OUT_DIR / "layerwise_pooling_ablation_top20.csv", index=False)
    print(best)
    print("Saved to", OUT_DIR)


if __name__ == "__main__":
    main()
```

Code listing: run_track1_spotcheck.py (1)

```
0001: import gc
0002: import os
0003: from pathlib import Path
0004:
0005: import numpy as np
0006: import pandas as pd
0007: from sklearn.linear_model import LogisticRegression
0008: from sklearn.metrics import accuracy_score
0009: from sklearn.pipeline import make_pipeline
0010: from sklearn.preprocessing import StandardScaler
0011: from sklearn.svm import LinearSVC
0012:
0013: import lie_llm_full_semantic_holdout_experiment as full
0014:
0015:
0016: OUT_DIR = Path(os.getenv("SPOTCHECK_OUT_DIR", "track1_spotcheck_results"))
0017: OUT_DIR.mkdir(parents=True, exist_ok=True)
0018:
0019: MODELS = [m.strip() for m in os.getenv("SPOTCHECK_MODELS", "bert-large-uncased").split(",") if m.strip()]
0020: N_BASE = int(os.getenv("SPOTCHECK_N_BASE", "100"))
0021:
0022:
0023: def run_classifiers(X, labels, split):
0024:     train_mask = split == "train"
0025:     test_mask = split == "test"
0026:     classifiers = {
0027:         "logreg": make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000, class_weight="balanced", n_jobs=-1)),
0028:         "linear_svc": make_pipeline(StandardScaler(), LinearSVC(class_weight="balanced", max_iter=20000)),
0029:     }
0030:     rows = []
0031:     for clf_name, clf in classifiers.items():
0032:         clf.fit(X[train_mask], labels[train_mask])
0033:         pred = clf.predict(X[test_mask])
0034:         rows.append({
0035:             "classifier": clf_name,
0036:             "accuracy": float(accuracy_score(labels[test_mask], pred)),
0037:             "n_train": int(train_mask.sum()),
0038:             "n_test": int(test_mask.sum()),
0039:         })
0040:     return rows
0041:
0042:
0043: def main():
0044:     full.OUT_DIR = str(OUT_DIR)
0045:     full.N_BASE = N_BASE
0046:     os.makedirs(full.OUT_DIR, exist_ok=True)
0047:
0048:     pair_df = full.build_dataset()
0049:     prompts, pair_df = full.index_prompts(pair_df)
0050:     pair_df.to_csv(OUT_DIR / "transformation_pairs.csv", index=False)
0051:     pd.DataFrame({"prompt_id": range(len(prompts)), "prompt": prompts}).to_csv(OUT_DIR / "prompts.csv", index=False)
0052:
0053:     labels = pair_df["class"].astype(str).to_numpy()
0054:     split = pair_df["split"].astype(str).to_numpy()
```

Code listing: run_track1_spotcheck.py (2)

```
0055:     src_idx = pair_df["source_idx"].to_numpy()
0056:     tgt_idx = pair_df["target_idx"].to_numpy()
0057:
0058:     rows = []
0059:     for model_name in MODELS:
0060:         print(f"\n=== MODEL: {model_name} ===")
0061:         vectors = full.get_vectors(model_name, prompts)
0062:         x_src = vectors[src_idx]
0063:         x_tgt = vectors[tgt_idx]
0064:         reps = {
0065:             "x_only": x_src,
0066:             "y_only": x_tgt,
0067:             "delta": x_tgt - x_src,
0068:             "concat": np.concatenate([x_src, x_tgt], axis=1),
0069:         }
0070:         for rep_name, X in reps.items():
0071:             for row in run_classifiers(X, labels, split):
0072:                 row.update({
0073:                     "model": model_name,
0074:                     "representation": rep_name,
0075:                     "random_baseline": 1 / len(set(labels)),
0076:                     "n_base": N_BASE,
0077:                 })
0078:                 print(model_name, rep_name, row["classifier"], f"{row['accuracy']:.4f}")
0079:                 rows.append(row)
0080:         del vectors, reps
0081:         gc.collect()
0082:
0083:     result = pd.DataFrame(rows)
0084:     result.to_csv(OUT_DIR / "spotcheck_representation_ablation.csv", index=False)
0085:     pivot = result.pivot_table(index=["model", "classifier"], columns="representation", values="accuracy").reset_index()
0086:     pivot.to_csv(OUT_DIR / "spotcheck_representation_ablation_pivot.csv", index=False)
0087:     print(pivot)
0088:
0089:
0090: if __name__ == "__main__":
0091:     main()
```